

skill:讓 AI 輸出規範易懂的人話

此 skill 是我個人整理的一套寫作規範, 來自對真實 AI 輸出中常見毛病的歸納, 用來把模型的輸出改寫成規範易懂的人話, 並能顯著減少 AI 味。內容涵蓋用詞表述、行文組織、舉例類比、排版等方面, 適用於 AI 講解、教學、寫作等場景。

此 skill 開源在 <https://github.com/Tsinswreng/skill-elucidate>。使用方式與一些注意要點在文末。本文先展示 skill 的正文內容, 正文內容如下:

1 | 使用規範的現代漢語書面語

1.1 | 表述要清晰

1.1.1 | 不要缺主語

錯誤示例:

✗ 點擊保存按鈕後, 會把修改保存到數據庫。

正確示例:

✓ 用戶點擊保存按鈕後, 系統會把修改保存到數據庫。

1.1.2 | 不要缺謂語

錯誤示例:

✗ 此項目名叫 MyProj, 個人項目。

正確示例:

✓ 此項目名叫 MyProj, 是個人項目。

1.1.3 | 指代要明確

避免濫用「他」「其」「這」「前/後者」等代詞, 尤其是在上下文中概念較多時。即使你做到了指代無歧義, 濫用代詞依然會增大讀者的閱讀負擔。應指明對象。

錯誤示例:

✗ 服務器收到請求後把它交給後端, 後端處理它, 再把它返回給前端。

正確示例:

✓ 服務器收到請求後把**請求**轉交給後端, 後端處理**該請求**, 再把**處理結果**返回給前端。

指代對象要指名詞, 讀者不用回頭猜「它」是誰。

1.1.4 | 說清限定語/說清事物的所屬

例:

1. 錯誤示例:

✗ 並發量增加之後響應時間顯著變長, 瓶頸大概率出在數據庫連接池不夠用, 建議先把上限調大再看。

上限到底是甚麼上限?

正確示例:

✓ 並發量增加之後響應時間顯著變長, 瓶頸大概率出在數據庫連接池不夠用, 建議先把**數據庫連接池的**上限調大再看。

2. 錯誤示例:

✗ 攻擊工作由大模型驅動, **下一條命令**不可預先審查。

唐突冒出「下一條命令」, 讓人不知所指

正確示例:

✓ 攻擊工作由大模型驅動: 攻擊命令是大模型現場生成的, 下一條命令在事前並不存在, 因此無法預先審查。

1.1.5 | 說清動作對象

如 回覆、傳達 等詞, 說清是誰給誰。

1.1.6 | 不使用不明確的縮寫/代號等

錯誤示例:

✗ MyProj 是某某比賽一等獎項目 (1/114514) 。

正確示例:

✓ MyProj 是某某比賽一等獎項目, 在 114514 個參賽項目中排名第一。

1.2 | 規範用詞

1.2.1 | 不要強行把雙字詞砍成單字

錯誤示例:

✗ 包間互引用 `workspace:*` 語法——引同倉成員、版本跟着走、不從 npm 拉。

正確示例:

✓ 包之間互相引用用的是 `workspace:*` 語法, 引用的就是同一倉庫裏的其他成員包, 依賴版本由倉庫內該包的當前版本決定, 不需要從 npm 拉取。

1.2.2 | 避免自造抽象術語

錯誤示例:

✗ 此模塊管容器的生滅

正確示例:

✓ 此模塊管理容器的創建與銷毀

1.2.3 | 除非術語或專門稱呼, 避免夾雜英文

錯誤示例:

✗ 調用方只依賴抽象, 不 care 內部實現細節

正確示例:

✓ 調用方只依賴抽象, 不關心內部實現細節

1.3 | 避免過度口語化

文檔正文要使用書面語, 不要把日常交談的口吻寫進正文。 錯誤示例:

✗ 這玩意負責把數據搞到數據庫裏去。

正確示例:

✓ 此組件負責把數據寫入數據庫。

1.4 | 避免憑空否定

先直接看例子:

問: 你是哪國人?

錯誤示例:

✗ 答: 我**不是**日本人, **不是**韓國人, **而是**中國人。

正確示例:

✓ 答: 我是中國人。

讓你介紹一樣東西時 你就直接說「他是甚麼」/「他怎麼樣」, 若沒人提到 xxx 你就不要憑空否定, 不要說「他不是 xxx 而是 yyy」。

僅在需要澄清真實的誤解或混淆項時纔使用, 禁止做無意義的[憑空捏造出一樣東西再去否定]。

判別標準: 被否定的那個理解必須是已經出現過的, 或確確實實容易引起混淆的。

例:

1.
 - 問: 我看你長得像東亞人, 你是日本人嗎?

- ☒ 答: 我不是日本人。我是中國人

2.

- 問: 軟刪除是甚麼
- 答: ☒ 軟刪除是把數據中的記錄標記為已刪除, 而不會真的物理刪數據的記錄 ☒ 先說是甚麼, 再否認易誤解項
- 答: ☒ 軟刪除不會真的物理刪數據的記錄, 而是把數據中的記錄標記為已刪除 ☒ 問甚麼就應該先答甚麼

1.5 | 禁止使用破折號

1.6 | 禁止濫用引號或括號作解釋說明

2 | 行文

2.1 | 注重邏輯連貫, 注重過渡

- 句子之間、段落之間的因果、順序、轉折關係要寫明, 不要讓讀者腦補。
- 用「先...再...」「若...則...」「因此」「另一方面」等連接詞點明關係。

錯誤示例:

✗ 服務器收到請求。緩存命中就返回。沒命中就查數據庫。數據庫也沒有就返回 404。

正確示例:

✓ 服務器收到請求後先查緩存。若緩存命中,則直接返回緩存結果;若未命中,則再查數據庫,若數據庫中也沒有則返回 404。

2.2 | 明確中心主旨/主題/寫作目的

文章邏輯要連貫,銜接自然,文章的每一部分都應為中心主旨服務,和寫作目的/寫作主題有關聯。不能想到甚麼就說甚麼。不要讓用戶疑惑「我明明在看關於 xxx 的文章,現在卻看的是 yyy 的內容,你寫的這些 yyy 和 xxx 有甚麼關係」

常見的病是相關內容放錯位置:開頭向讀者許諾一條主線,中途整段鋪開通用背景,進出都沒有交代,讀者一到那裏就迷路。

錯誤示例:

✗ 本文跟着一個網頁請求走一遍: 從你按下回車, 到頁面顯示在眼前。你在地址欄輸入 `example.com`, 按下回車。瀏覽器不知道這個名字對應哪臺服務器, 第一件事是查它的地址。

DNS 誕生於 1983 年, 設計目的是取代手工維護的主機表。DNS 採用分層結構, 根域、頂級域、權威域逐級授權。查詢方式分兩種: 遞歸查詢把問題整個交給服務器代辦; 迭代查詢則是逐層問路, 答案自己彙總。記錄類型也多樣, A 記錄、CNAME 記錄、MX 記錄各有用途。

查到地址後, 瀏覽器與服務器建立連接。

讀者讀到中段便會問: 「我明明在看這次請求的旅程, 現在讀的是 DNS 的通用介紹, 這些內容和這次請求有甚麼關係?」

正確示例:

✓ 本文跟着一個網頁請求走一遍: 從你按下回車, 到頁面顯示在眼前。你在地址欄輸入 `example.com`, 按下回車。瀏覽器不知道 `example.com` 的服務器在哪, 第一件事是查它的地址, 這一步叫域名解析。瀏覽器先翻自己的緩存, 緩存裏沒有, 就把名字交給本地 DNS 服務器; 本地服務器也不知道, 就

逐層向上問,直到有服務器給出答案。地址到手,瀏覽器與服務器建立連接。

同樣講了 DNS, 由「瀏覽器不知道服務器在哪」把它帶出來, 只講這次請求會用到的部分, 講完用「地址到手」送回主線。

3 | 多舉例子 結合例子說明

示例:

問: 等差數列怎麼求和?

錯誤示例:

✗ 答: 首項加末項乘項數除以 2。

正確示例:

✓ 答: 首項加末項乘項數除以 2。例如 2, 4, 6, 8, 10 這個等差數列, 該數列的首項為 2, 末項為 10, 因為一共有 5 項故項數為 5, 故該數列的和為:

$$(2+10)*5/2=30$$

4 | 類比與比喻

- 僅在適當的時間使用, 不要濫用。
- 濫用的比喻、不恰當的比喻增加理解成本、降低深度。
- 類比要恰當, 多用同類的事物來類比, 少用抽象的比喻。

錯誤示例:

✗ 垃圾回收就像一個勤勞的清潔工每天打掃衛生。

正確例:

✓ C# 的 `Task` 就相當於 js 的 `Promise` ✓ 恰當的類比

5 | 排版

文章除了內容上要易懂之外, 在排版上也要下功夫, 讓人看得舒服。

5.1 | 注意分行分段

該分行分段就分, 避免一路小句寫到尾。

錯誤示例:

✗ 緩存用來減少數據庫的壓力。讀取數據時先查緩存, 命中就直接返回; 未命中才查數據庫, 並把結果寫回緩存。緩存也有代價。它佔用內存, 而且數據可能過期, 所以要設置失效策略, 否則會讀到舊值。

正確示例:

✓ 緩存用來減少數據庫的壓力。讀取數據時先查緩存, 若命中就直接返回; 未命中纔查數據庫, 並把結果寫回緩存。

緩存也有代價。它佔用內存, 而且數據可能過期, 所以要設置失效策略, 否則會讀到舊值。

5.2 | 善用列表與加粗

可採用列表、加粗等語法讓文章更有層次

錯誤示例:

✗ 其一, xxx; 其二, yyy

正確示例:



1. xxx
2. yyy

5.3 | 引文 要寫明 突出

錯誤示例:

- ✗ 快照的內容大意是： 項目名:「juice-shop」 ; 零個攻擊手在運行; 並發上限是七; 可用的攻擊手提示詞只有一份, 名字叫 tsinswreng-security 模型能看見的只有這段發過去的文字, 系統內部存了什麼數據, 模型一無所知。

正確示例:

- ✓ 快照的內容大意是:
 - ▶ 項目名:「juice-shop」 ;
 - ▶ 正在運行的攻擊手: 0 個;
 - ▶ 併發上限: 7;
 - ▶ 可用的攻擊手提示詞: 只有一份, 名字叫 tsinswreng-security

模型能看見的只有這段發過去的文字，系統內部存了什麼數據，模型一無所知。

注: 如果你使用 typst, 在 typst 中, 引用格式語法為:

```
#quote(block:true)[  
  引文內容  
]
```

typst

6 | 綜合示例

1. 錯誤示例

✗ 此工作區是 MyProj——面向某某場景的多 Agent 協作平臺, Bun monorepo。某某比賽一等獎項目 (1/114514)。公司項目, 雙倉庫: GitHub 開源側 `company-org/MyProj` (公司 org); 公司內部側 `ComMyProj` (gitlab 託管、非 GitHub, 歷史與開源側完全獨立, 最新改動都在此。用戶是公司員工、接手負責開發)。主開發場 (2026-09-18 起): 工作區內 `ComMyProj` 的 `dev` 分支

正確示例

✓ 此工作區屬於 MyProj 項目。MyProj 是面向某某場景的多 Agent 協作平臺，採用 Bun 單體倉庫結構。MyProj 在某某比賽中獲得一等獎，在 114514 個參賽項目中排名第一。

MyProj 是公司項目，對應兩個遠端倉庫：

- GitHub 開源側的倉庫名為 `company-org/MyProj`，為公司的開源倉庫；
- 公司內部側的倉庫名為 `ComMyProj`，它託管在公司內部的 gitlab 上；與 GitHub 開源側沒有共同的提交歷史。用戶是公司員工，接手負責開發。

公司的最新改動都在公司內部側倉庫，因此從 2026-09-18 起，主開發場是工作區內 ComMyProj 的 `dev` 分支。

2. 錯誤示例

✗ 起容器時任務書隨啓動命令送進去。之後容器裏的每一步經 rpc 實時流回宿主。為甚麼分這兩層：動手的活是模型驅動的，下一條命令不可預先審查，不能讓它直接在宿主上跑；一個攻擊手一個容器，誰

把環境打掛了不牽連別人和宿主；環境一次性，裝過什麼、留下什麼垃圾都隨容器消失。

正確示例

✓ 容器啓動時，任務書隨啓動命令送進容器。之後容器裏的每一步都經由 rpc 實時流回宿主。爲什麼要分成宿主進程與容器兩層：因爲攻擊工作由大模型驅動，攻擊命令由大模型當場生成，大模型輸出的下一條命令無法預先審查，難免混有破壞性的操作，所以這樣的命令不能直接在宿主上運行。一個攻擊手配一個容器。即使一個容器裏的環境損壞，也不會牽連其他容器和宿主。環境是一次性的。容器裏安裝過什麼、留下什麼垃圾，都隨着容器一起消失。

3. 錯誤示例

✗ 派工決策到了 `runtime` 手裏才落地：構建鏡像、起容器、回收容器。一個攻擊手一個容器，一次任務一個隔離沙箱：打掛互不影響，環境用完即拋。起容器當場，宿主把任務書拼好、隨啓動命令送進去。任務書是中文模板，拼五樣：項目名與描述；目標入口清單，

每條帶指紋； 情報池兩塊看板與已交報告的三段摘要； 行為要求清單； 最後把開工便條以「最高優先級遵循」附在末尾。

正確示例

✓ 派工決策由 `runtime` 執行, `runtime` 負責做三件事: 構建鏡像、創建容器、回收容器。一個攻擊手配一個容器, 一次任務就是一個隔離沙箱: 各容器的環境損壞互不影響, 環境用完隨容器一起丟棄。容器起動時, 宿主把組裝好的任務書隨啓動命令送進容器。任務書是中文模板, 由五部分組成:

1. 項目名與描述
2. 目標入口清單, 每條帶有指紋
3. 情報池兩塊看板與已交報告的三段摘要
4. 行為要求清單
5. 開工便條, 附在任務書末尾, 標着最高優先級

7 | 重構表述

重構表述 即 把有問題的文本改寫成表述清晰的文本。

重構表述的要點:

- 逐條檢查是否滿足上述要求
- 站在人類讀者的角度審視是否易懂? 是否有問題?
- 重構後的文本是否缺少必要的信息量?

7.1 | 重構不應減少有效信息量

重構表述只是換一種表述方法, 重構後該有的有用信息量應該保持不變。
對於原文中不必要的冗餘的內容, 纔可以考慮刪除。

8 | 自查

問自己兩個問題:

- 是否滿足了上述要求:
- 站在人類讀者的角度 是否易懂? 是否有問題?

-----skill 正文到此結束-----

1 | 安裝

找到 github 倉庫中的 `SKILL.md` ,

- 簡中: <https://github.com/Tsinswreng/skill-elucidate/blob/master/Langs/zh-Hans/tsinswreng-elucidate/SKILL.md>
- 繁中: <https://github.com/Tsinswreng/skill-elucidate/blob/master/Langs/zh-Hant/tsinswreng-elucidate/SKILL.md>

手動放到項目的 skill 目錄下(如 `.agent/skills/tsinswreng-elucidate/`)。該 skill 的默認名稱是 `tsinswreng-elucidate` , 其中 `tsinswreng-` 是本人名稱, 充當命名空間, 無實際意義。

您可自行按需更改該 skill 任何部分的內容, 不限于名稱、描述、正文等, 反正此 skill 就是一段自然語言文本。

2 | 使用例

這裏展示一個[使用該 skill 來潤色文本]的實例。使用的模型是 glm-5.2-highspeed。以下是未使用此 skill 時模型輸出的一段文字, 來自在某項目中 AI 對遇到的一個 bug 的總結, 內容完全由模型自己發揮:

OpenCode 版本必須精確 pin(當前 1.18.23)並端到端驗證: Docker Hub 的 `openeuler/opencode` 標籤和 npm 的 `opencode-ai` 是兩套版本; 歷史上“版本

不低於下限”的策略造成開發/生產漂移, token 統計全壞。禁止用可變 `latest`。

可見無約束時, AI 直接輸出的內容非常符合 AI 的文風, 且看起來不是這麼清晰易懂。隨後要求大模型按要求改寫潤色, 人工發送以下提示詞給大模型:

看 elucidate skill, 改寫這段話。

改寫後的一次產出:

OpenCode 的版本必須固定到具體的版本號, 當前固定的版本是 1.18.23; 固定之後還要對版本做端到端驗證。這條規定的依據如下:

1. Docker Hub 上 `openeuler/opencode` 鏡像的標籤與 npm 上 `opencode-ai` 包的版本號是兩套互不相干的版本體系, 確認版本時必須兩邊都查。
2. 歷史上採用過「版本不低於下限」的策略, 也就是只要求版本號達到某個下限, 不鎖定具體版本。這個策略造成開發環境與生產環境所用的版本發生漂移, 最終 token 統計功能完全損壞。
3. `latest` 是可變標籤, `latest` 指向的版本會隨時間變化, 因此禁止使用 `latest`。

可見, 該 skill 還是有一定效果的。

3 | 幾個注意點

1. 建議先讓 AI 直接輸出,再讓 AI 按照 skill 修改。相較於直接讓 AI 按照 skill 規則一次性輸出完,這樣做的話,「初始輸出」與「按 skill 修改潤色」被人為分成了兩個任務,效果往往會更好。

原因在于模型的注意力有限。生成時,模型的注意力要分配到上下文的全部內容上,上下文裏的規約越多,每條規約分到的注意力越少,模型越容易漏掉上下文中間的內容。所以分兩遍做:第一遍讓模型寫出初稿,第二遍讓模型拿規約逐條對照初稿修改。這樣第二遍只有對照與修改一件事,每條規約才更容易被遵守執行。

2. 此 skill 不是銀彈,產出的文本的品質模型能力有關。一次性修改未必能使您完全滿意,可適當放低期待,讓 AI 按 skill 多輪對照再修改,不必指望 AI 能一次性改好。通常每輪潤色修改後的產出品質都會有提升。